```python
def main():
    print("Hello, World!")

    x = 10
    y = "Python"

    print("Hello, and world"))
```

آغاز برنامه‌نویسی با پایتون

متغیرها و انواع داده‌های پایه

```python
    x = 10
    y = "Python"

class DataProcessor:
    def __init__(self):
        pass
```

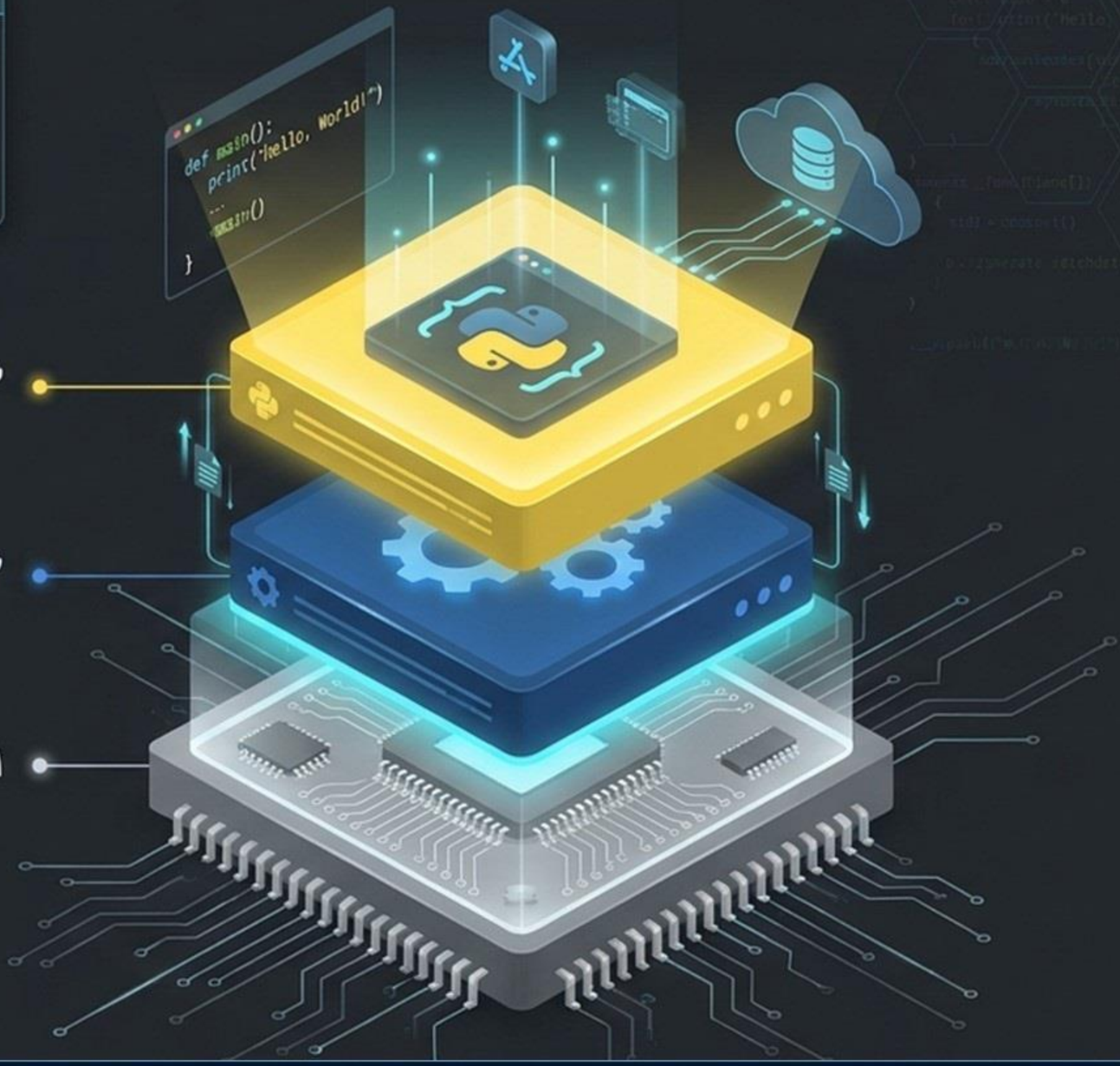
جایگاه پایتون در معماری کامپیوتر (Recap)

چگونه از صفر و یک به پایتون رسیدیم؟

۳. **پایتون:** زبان سطح بالا برای فرمان دادن به سیستم

۲. **سیستم‌عامل:** مدیر منابع سیستم

۱. **سخت‌افزار:** مغز فیزیکی (CPU, RAM)



اهداف یادگیری (Learning Objectives)

در پایان این جلسه شما قادر خواهید بود:

تشریح (Explain):
درک نحوه کار مفسر پایتون.

پیاده‌سازی (Implement):
استفاده از دستورات ورودی و خروجی (`print`, `input`).

تعریف (Define):
ایجاد متغیرها و رعایت استانداردهای نام‌گذاری.

شناخت (Identify):
تشخیص انواع داده‌های پایه و تبدیل آن‌ها (Type Casting).

چرا پایتون؟

زبان محبوب غول‌های فناوری



- سینتکس خوانا: بسیار نزدیک به زبان انگلیسی.
- تمرکز بر حل مسئله: عدم درگیری با مدیریت پیچیده حافظه.
- مفسری بودن (Interpreted): اجرای خط به خط و عیب‌یابی سریع.
- کاربرد: هوش مصنوعی، وب، اسکریپت‌نویسی.



متغیرها (Variables) چیستند؟

متغیر مثل یک **جعبه** در انبار **حافظه** (RAM) است که روی آن برچسب می‌چسبانیم.

بدون نیاز به تعیین نوع صریح (Dynamic Typing):



قوانین نام‌گذاری (Naming Conventions)

✅ مجاز و استاندارد (snake_case):

```
student_score = 20
user_age_2 = 25
```

❌ غیرمجاز و خطا:

```
2age = 25
```
(شروع با عدد ممنوع)

```
for = 10
```
(استفاده از کلمات کلیدی ممنوع)

⚠️ هشدار: حساسیت به حروف (Case Sensitive) - متغیر Age با age کاملا متفاوت است.

انواع داده‌های پایه (Basic Data Types)

۱. عدد صحیح (int): اعداد بدون اعشار

```
10, -3, 0
```

۲. عدد اعشاری (float): اعداد با نقطه اعشار

```
3.14, -0.001
```

۳. رشته متنی (str): کاراکترها داخل کوتیشن

```
"Hello", "1405"
```

۴. ارزش منطقی (bool): فقط درست یا غلط

```
True, False
```

ارتباط با ماشین: خروجی و ورودی

آشنایی با توابع print (صدای برنامه) و input (گوش برنامه).

خروجی (Print):

```
print('Hello, Python!')
```

```
> Hello, Python!
```

ورودی (Input):

```
user_name = input('Enter name: ')
print('Welcome', user_name)
```

```
> Enter name: Ali
> Welcome Ali
```

چرا '5' + '5' مساوی 10 نمی‌شود؟

رفتار رشته‌ها (str): `'5' + '5'` → `'55'`

رفتار اعداد (int): `5 + 5` → `10`



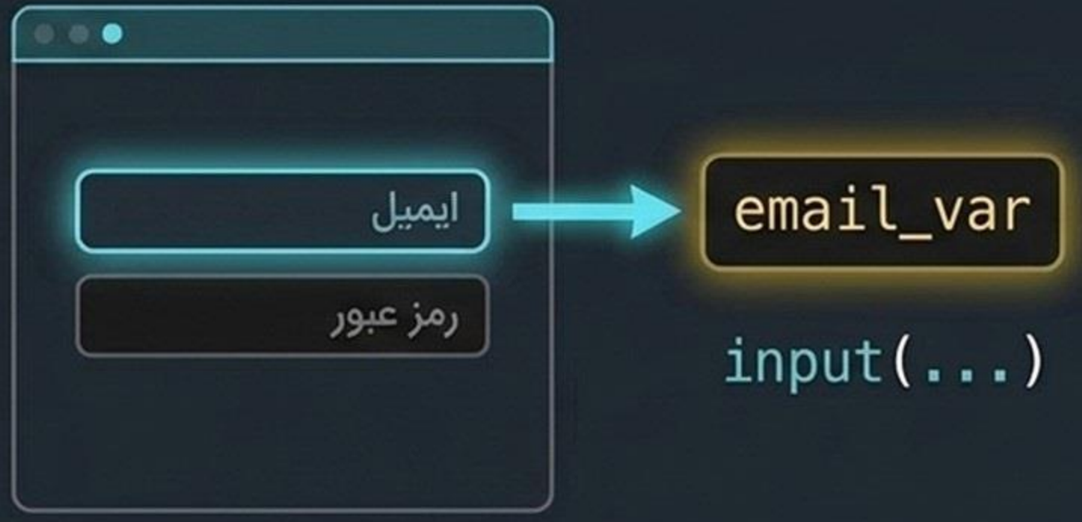

```
int('5') -> 5
float(10) -> 10.0
str(2026) -> '2026'
```

ارتباط با دنیای واقعی

متغیرها و ورودی‌ها در صنعت کجا کاربرد دارند؟

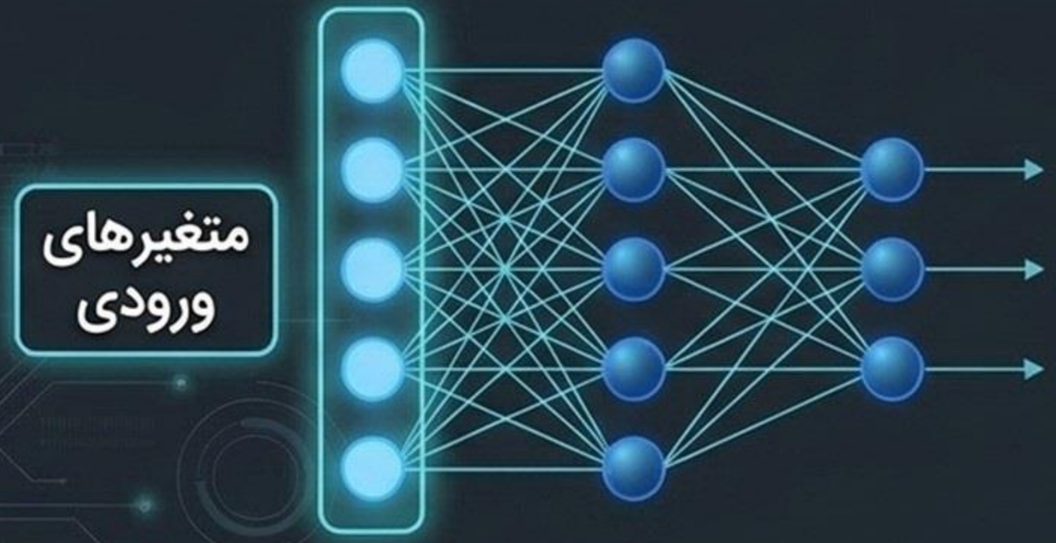
توسعه وب و اپلیکیشن

فیلدهای فرم ثبت‌نام (ایمیل، رمز عبور) با دستور input دریافت و در متغیرها ذخیره می‌شوند.



هوش مصنوعی و ML

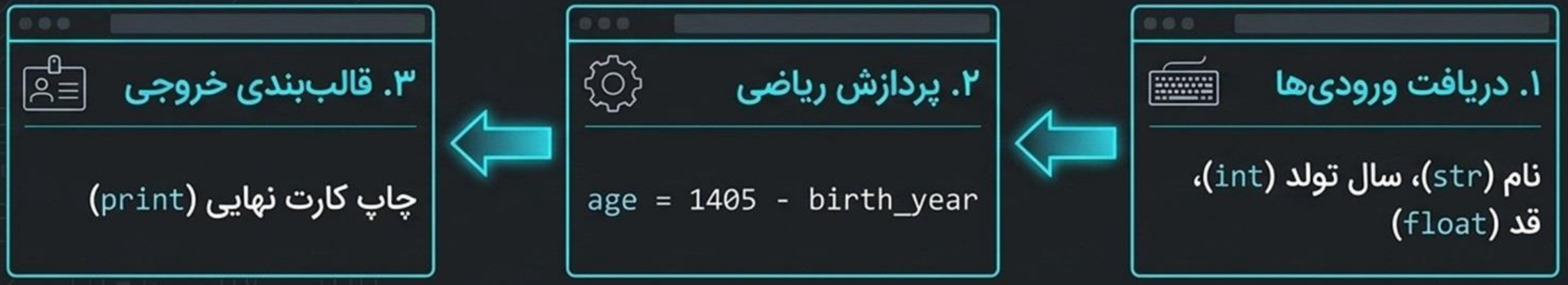
دیتاست‌های حجیم به عنوان متغیر وارد مدل‌های ریاضی می‌شوند تا پس از پردازش، خروجی تولید کنند.



پروژه عملی (Live Coding): معماری سیستم

کارت شناسایی دیجیتال (Digital ID Card)

صورت مسئله: دریافت اطلاعات کاربر، محاسبه سن برای سال ۱۴۰۵، و چاپ کارت نهایی.



پیاده‌سازی: کدهای پروژه و خروجی ترمینال

main.py

```python
name = input('نام: ')
birth_year = int(input('سال تولد: '))
height = float(input('قد (متر) '))

age = 1405 - birth_year

print(f'--- کارت شناسایی ---\n')
print(f'نام:{name}| سن تخمینی:{age} | قد:{height}')
```

```
> نام: علی
> سال تولد: ۱۳۸۳
> قد (متر): ۱.۷۵

> --- کارت شناسایی ---
> نام: علی | سن تخمینی: ۲۲ | قد: ۱.۷۵
```

تمرین و چالش خارج از کلاس

مینی‌پروژه: ماشین‌حساب مالیاتی

۱. حقوق ماهانه را بگیرید (**float**).
۲. مالیات ثابت ۱۰٪ را محاسبه کنید.
۳. حقوق خالص نهایی را به دست آورید.
۴. خروجی را با قالب‌بندی زیبا چاپ کنید.

چالش مهندسی (Advanced)

دو عدد را به صورت رشته (**str**) بگیرید، به **float** تبدیل کنید، ۴ عمل اصلی ریاضی را انجام دهید و با تابع **type** نوع نتیجه را چاپ.

مرور سریع (Key Takeaways)

آنچه امروز در ساختار پایتون کشف کردیم:

- **متغیرها:** خانه‌های حافظه بدون نیاز به تعیین نوع صریح (Dynamic Typing).
- **انواع داده:** اعداد (int, float)، متن (str)، منطق (bool).
- **ورودی کاربر:** تابع (input) به صورت پیش‌فرض رشته متنی (str) برمی‌گرداند.
- **تبدیل (Casting):** برای محاسبات ریاضی، تبدیل ورودی‌ها به عدد الزامی است.

جلسه آینده: جریان کنترل و تصمیم‌گیری (if, else)